Name:Shoeb Sheikh.USN No.:CS23O74

Aim 3_Extra:To implement Logistic Regression for binary or multiclass classification problems and analyze the classification performance.

```
import seaborn as sns
import pandas as pd
import numpy as np
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
path = '/content/drive/MyDrive/ML_Classes/diabetes_prediction_dataset.csv'
df = pd.read_csv(path)
df.head()
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
0	Female	80.0	0	1	never	25.19	6.6	140	0
1	Female	54.0	0	0	No Info	27.32	6.6	80	0
2	Male	28.0	0	0	never	27.32	5.7	158	0
3	Female	36.0	0	0	current	23.45	5.0	155	0
4	Male	76.0	1	1	current	20.14	4.8	155	0

Next steps: Generate code with df New interactive sheet

```
df.describe()
```

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose_level	diabetes
count	100000.000000	100000.00000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
mean	41.885856	0.07485	0.039420	27.320767	5.527507	138.058060	0.085000
std	22.516840	0.26315	0.194593	6.636783	1.070672	40.708136	0.278883
min	0.080000	0.00000	0.000000	10.010000	3.500000	80.000000	0.000000
25%	24.000000	0.00000	0.000000	23.630000	4.800000	100.000000	0.000000
50%	43.000000	0.00000	0.000000	27.320000	5.800000	140.000000	0.000000
75%	60.000000	0.00000	0.000000	29.580000	6.200000	159.000000	0.000000
max	80.000000	1.00000	1.000000	95.690000	9.000000	300.000000	1.000000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   gender               100000 non-null  object
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  object
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB
```

```
df.shape
```

```
(100000, 9)
```

```
df
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
0	Female	80.0	0	1	never	25.19	6.6	140	0
1	Female	54.0	0	0	No Info	27.32	6.6	80	0
2	Male	28.0	0	0	never	27.32	5.7	158	0
3	Female	36.0	0	0	current	23.45	5.0	155	0
4	Male	76.0	1	1	current	20.14	4.8	155	0
...	...	...	...	...	...	...	...	...	...
99995	Female	80.0	0	0	No Info	27.32	6.2	90	0
99996	Female	2.0	0	0	No Info	17.37	6.5	100	0
99997	Male	66.0	0	0	former	27.83	5.7	155	0
99998	Female	24.0	0	0	never	35.42	4.0	100	0
99999	Female	57.0	0	0	current	22.43	6.6	90	0

100000 rows × 9 columns

Next steps: Generate code with df New interactive sheet

```
df['smoking_history'].unique()
```

```
array(['never', 'No Info', 'current', 'former', 'ever', 'not current'],
      dtype=object)
```

```
df.isnull().sum()
```

	0
gender	0
age	0
hypertension	0
heart_disease	0
smoking_history	0
bmi	0
HbA1c_level	0
blood_glucose_level	0
diabetes	0

dtype: int64

```
#remove never
df=df[df['smoking_history']!='never']
```

```
df.head()
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
1	Female	54.0	0	0	No Info	27.32	6.6	80	0
3	Female	36.0	0	0	current	23.45	5.0	155	0
4	Male	76.0	1	1	current	20.14	4.8	155	0
7	Female	79.0	0	0	No Info	23.86	5.7	85	0
11	Female	54.0	0	0	former	54.70	6.0	100	0

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(64905, 9)
```

```
df['smoking_history'].unique()
```

```
array(['No Info', 'current', 'former', 'ever', 'not current'],
      dtype=object)
```

```
df['smoking_history']= df['smoking_history'].map({'not current':0 , 'ever':1, 'No Info':2, 'current':3, 'former':4})
```

```
/tmp/ipython-input-260096443.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df['smoking_history']= df['smoking_history'].map({'not current':0 , 'ever':1, 'No Info':2, 'current':3, 'former':4})
```

```
df.head()
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level	diabetes
1	Female	54.0	0	0	NaN	27.32	6.6	80	0
3	Female	36.0	0	0	NaN	23.45	5.0	155	0
4	Male	76.0	1	1	NaN	20.14	4.8	155	0
7	Female	79.0	0	0	NaN	23.86	5.7	85	0
11	Female	54.0	0	0	NaN	54.70	6.0	100	0

Next steps: Generate code with df New interactive sheet

```
#split dataset into independent and dependent
X=df.iloc[:,:-1]
print(X)
```

```
       gender   age  hypertension  heart_disease  smoking_history    bmi  \
1      Female  54.0             0              0              NaN  27.32
3      Female  36.0             0              0              NaN  23.45
4        Male  76.0             1              1              NaN  20.14
7      Female  79.0             0              0              NaN  23.86
11     Female  54.0             0              0              NaN  54.70
...       ...   ...           ...            ...              ...    ...
99994  Female  36.0             0              0              NaN  24.60
99995  Female  80.0             0              0              NaN  27.32
99996  Female   2.0             0              0              NaN  17.37
99997    Male  66.0             0              0              NaN  27.83
99999  Female  57.0             0              0              NaN  22.43

       HbA1c_level  blood_glucose_level
1              6.6                   80
3              5.0                  155
4              4.8                  155
7              5.7                   85
11             6.0                  100
...            ...                  ...
99994          4.8                  145
99995          6.2                   90
99996          6.5                  100
99997          5.7                  155
99999          6.6                   90

[64905 rows x 8 columns]
```

```
y=df.iloc[:,-1]
print(y)
```

```
1        0
3        0
4        0
7        0
11       0
        ..
99994    0
99995    0
99996    0
99997    0
99999    0
Name: diabetes, Length: 64905, dtype: int64
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                    test_size =0.3, random_state=42)
```

```
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression()
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Process categorical features using one-hot encoding
# 'gender' is still categorical, and 'smoking_history' might have NaNs or be object type in X_train/X_test
X_train_processed = pd.get_dummies(X_train, columns=['gender', 'smoking_history'], drop_first=True)
X_test_processed = pd.get_dummies(X_test, columns=['gender', 'smoking_history'], drop_first=True)

# Ensure columns are aligned after one-hot encoding
X_test_processed = X_test_processed.reindex(columns=X_train_processed.columns, fill_value=0)

# Fit the classifier on the processed training data
classifier.fit(X_train_processed, y_train)

# Make predictions on the processed test data
y_pred = classifier.predict(X_test_processed)

# accuracy score
score= accuracy_score(y_test, y_pred)
print(score)
```

Show hidden output

```
#accuracy score
score= accuracy_score(y_test, y_pred)
print(score)
```

```
0.9647699260476582
```

```
#for precision, recall,F1 score & accuracy
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98     18001
           1       0.87      0.63      0.73      1471

    accuracy                           0.96     19472
   macro avg       0.92      0.81      0.85     19472
weighted avg       0.96      0.96      0.96     19472
```

```
sns.pairplot(df, hue='smoking_history')
```

Show hidden output